



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Computação Gráfica

Trabalho Prático

Grupo 15

Edgar Ferreira	Fernando Pires	Sara Silva	Zita Duarte
a99890	a77399	a104608	a104268

Data da Receção	
Responsável	
Avaliação	
Observações	

Computação Gráfica

Edgar Ferreira

a99890

Fernando Pires

a77399

Sara Silva

a104608

Zita Duarte

a104268

março, 2025

Índice

1. Introdução	1
2. <i>Engine</i>	2
2.1. Renderização da Cena	2
2.1.1. Leitura e Interpretação do XML	2
2.1.2. Transformações	3
2.1.2.1. Translações	4
2.1.2.2. Rotações	4
2.1.2.3. Escalas	5
2.1.3. Renderização dos grupos	5
3. Testes	7
4. Scripts	9
5. Conclusões	10

Lista de Figuras

Figura 1	Gráfico representativo da aplicação de uma translação a um ponto	4
Figura 2	Gráfico representativo da aplicação de uma rotação a um ponto	4
Figura 3	Gráfico representativo da aplicação de uma mudança de escala a um ponto	5
Figura 4	Teste 1 - Cubo com translação	7
Figura 5	Teste 2 - Cubo, cone e esfera com translações	7
Figura 6	Teste 3 - Duas esferas e um cone com translações, escalas e rotações	7
Figura 7	Teste 4 - Cubos com translações e rotações	7
Figura 8	Renderização da cena do sistema solar	8

1. Introdução

Este relatório foi realizado no âmbito da Unidade Curricular de **Computação Gráfica** (CG) no ano letivo 2024/2025 e descreve o desenvolvimento da **segunda fase do projeto**. Nesta fase, foi implementada uma **estrutura hierárquica** para a representação de grupos com os modelos e transformações geométricas sobre os mesmos, permitindo a criação de cenas mais complexas como o **Sistema Solar**.

2. Engine

Na primeira fase, a estrutura era baseada apenas numa lista de modelos independentes, sem qualquer hierarquia ou influência entre eles. Nesta segunda fase, o modelo foi alterado para um sistema de grupos, onde cada grupo pode conter transformações próprias e influenciar seus subgrupos e modelos associados. Esta diferenciação possibilita maior flexibilidade e organização na construção de cenas mais complexas.

- **Grupo de modelos** (*group*), organizados de forma hierárquica, podendo conter:
 - **Transformações geométricas** (*translate*, *rotate*, *scale*), aplicadas a todos os elementos do grupo;
 - **Modelos** (*model*), representados pelos ficheiros `.3d` ou `.obj` já conhecidos da primeira fase;
 - **Subgrupos**, permitindo a criação de estruturas aninhadas.

A estrutura hierárquica introduzida nesta fase permite que transformações aplicadas a um grupo afetem todos os seus subgrupos e modelos, garantindo uma abordagem modular e reutilizável.

2.1. Renderização da Cena

2.1.1. Leitura e Interpretação do XML

Para processar os novos grupos e suas transformações, é necessário que o **Engine** realize a leitura do ficheiro *XML* e interprete sua estrutura de forma hierárquica. Cada grupo presente no *XML* segue a seguinte organização:

```
<group>
  <transform>
    <!-- Transformações, como translate, rotate, scale -->
  </transform>
  <models>
    <!-- Modelos -->
  </models>
  <group> <!-- Grupo filho -->
    <!-- Outros subgrupos -->
  </group>
</group>
```

As informações extraídas do grupo no ficheiro *XML* são agora armazenadas na classe `Group`, que representa a **estrutura hierárquica** de transformações, modelos e subgrupos. Essa classe permite organizar a cena de forma **modular**, garantindo que as transformações sejam aplicadas corretamente a todos os elementos pertencentes ao grupo.

```
class Group {
public:
    std::vector<Transform> transforms;
    std::vector<Model> models;
    std::vector<Group> children;

    Group();

    void addTransform(const Transform& transform);
    void addModel(const Model& model);
    void addChild(const Group& child);
    void draw();
};
```

Cada grupo pode conter transformações, modelos e subgrupos. Durante o *parsing* do XML, o motor percorre a sua estrutura recursivamente, garantindo que as transformações aplicadas a um grupo sejam herdadas pelos seus subgrupos.

Sendo assim, a classe principal com as configurações de renderização `Configuration` necessitou de ser alterada, ficando encarregue por armazenar o grupo global e seus subgrupos, em vez de apenas uma lista de caminhos para os modelos a construir.

2.1.2. Transformações

Nesta fase, cada grupo possui um conjunto de transformações geométricas (*translate*, *rotate* e *scale*) aplicadas de maneira **hierárquica**. Isso significa que os subgrupos herdam as transformações dos grupos superiores.

Cada tipo de transformação é armazenada como uma classe `Transform` dentro do conjunto de transformações do grupo, que mantém os valores necessários para a **translação**, **rotação** ou **escala** dependendo da transformação pedida no *XML*.

```
class Transform {
public:
    float translate[3] = {0, 0, 0};
    float rotate[4] = {0, 0, 0, 0};
    float scale[3] = {1, 1, 1};

    Transform() {}
};
```

Cada transformação pode ser especificada dentro de um nó `<transform>` no ficheiro *XML*. O nó pode conter um ou mais tipos de transformação, e a ordem delas deve ser respeitada durante a aplicação. Por exemplo, o seguinte *XML* define uma rotação seguida por uma escala:

```
<transform>
  <rotate angle="90" x="0" y="0" z="1" />
  <scale x="0.5" y="2" z="1" />
</transform>
```

Aqui, a rotação de 90 graus ao longo do eixo `Z` será aplicada primeiro, seguida pela escala nos eixos `X`, `Y` e `Z`. A ordem das operações é crucial para o efeito final, pois as transformações de escala, rotação e translação podem alterar os resultados de forma diferente dependendo da sequência em que são aplicadas.

2.1.2.1. Translações

As translações deslocam o grupo ao longo dos eixos X, Y e Z.

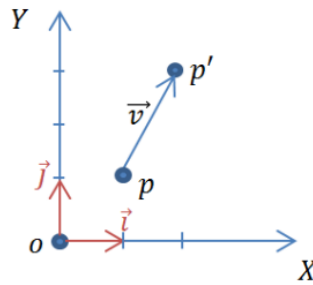


Figura 1: Gráfico representativo da aplicação de uma translação a um ponto

Na imagem, o ponto p é deslocado pelo vetor de translação v , resultando no novo ponto p' . Isso significa que todas as coordenadas do objeto são somadas às componentes do vetor de translação (tx, ty, tz) , ou seja:

$$P' = (x + tx, y + ty, z + tz)$$

No *OpenGL*, a translação é aplicada com a função `glTranslatef(dx, dy, dz)`, que multiplica a matriz de transformação atual pelo vetor de translação.

No XML, a translação é definida como: `<translate x="tx" y="ty" z="tz" />`

2.1.2.2. Rotações

As rotações giram um grupo em torno de um eixo especificado.

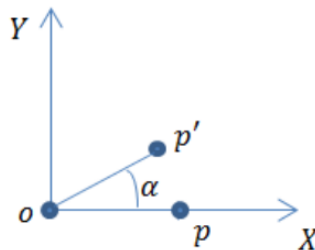


Figura 2: Gráfico representativo da aplicação de uma rotação a um ponto

Na imagem, o ponto p é rotacionado de um ângulo α ao redor da origem, resultando no novo ponto p' . No *OpenGL*, a rotação é aplicada com a função `glRotatef(angle, rx, ry, rz)`, que aplica uma matriz de rotação ao objeto.

Quando rotacionamos um ponto em torno do eixo Z ($rx = 0, ry = 0, rz = 1$), por exemplo, as suas coordenadas (x, y, z) são transformadas da seguinte forma:

$$x' = x \cdot \cos(\text{angle}) - y \cdot \sin(\text{angle})$$

$$y' = x \cdot \sin(\text{angle}) + y \cdot \cos(\text{angle})$$

$$z' = z$$

A mesma lógica aplica-se a rotações em torno dos eixos X e Y.

No XML, a rotação é definida como: `<rotate angle="a" x="rx" y="ry" z="rz" />`

2.1.2.3. Escalas

As escalas alteram o tamanho do objeto ao longo dos eixos X , Y e Z . Para aplicar as escalas, fizemos uso da função `glScalef(sx, sy, sz)` do *OpenGL*, que multiplica cada coordenada por esses fatores definidos, ou seja, cada ponto do modelo é atualizado como:

$$P' = (x \cdot sx, y \cdot sy, z \cdot sz)$$

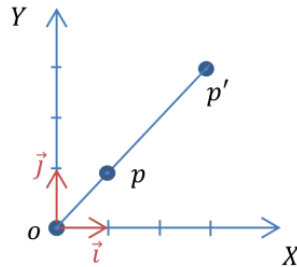


Figura 3: Gráfico representativo da aplicação de uma mudança de escala a um ponto

Na imagem, o ponto p é escalado, resultando no novo ponto p' , que pode estar mais afastado ou mais próximo da origem no novo referencial, dependendo dos fatores de escala aplicados.

No XML, a escala é definida como: `<scale x="sx" y="sy" z="sz" />`

2.1.3. Renderização dos grupos

A renderização dos grupos e seus elementos é realizada pela função `draw`, que aplica as transformações do grupo antes de desenhar seus modelos e subgrupos. As transformações são aplicadas na ordem em que foram definidas, utilizando as funções *OpenGL* já referidas (`glTranslatef`, `glRotatef`, `glScalef`), que modificarão automaticamente as matrizes alvo de acordo com o esperado.

```
void Group::draw() {
    glPushMatrix();

    // Transformações
    for (const Transform& transform : transforms) {
        glTranslatef(transform.translate[0], transform.translate[1],
transform.translate[2]);
        glRotatef(transform.rotate[0], transform.rotate[1], transform.rotate[2],
transform.rotate[3]);
        glScalef(transform.scale[0], transform.scale[1], transform.scale[2]);
    }

    // Modelos
    for (Model& model : models) {
        model.draw();
    }

    // Filhos
    for (Group& child : children) {
        child.draw();
    }

    glPopMatrix(); // Reset
}
```

Ao projetar o sistema de transformações no *OpenGL*, consideramos como cada transformação (translação, rotação ou escala) afeta o sistema de coordenadas do objeto. Cada transformação modifica o referencial do objeto, e isso influencia as transformações subsequentes. Ou seja, quando realizamos, por exemplo, uma rotação, os eixos do objeto são ajustados com base nessa rotação. Como resultado, todas as operações

subsequentes (como uma nova rotação ou translação) serão realizadas nesse novo sistema de coordenadas modificado, e não no sistema de coordenadas original. Este comportamento é fundamental para o funcionamento de hierarquias de grupos no *OpenGL*, pois ele assegura que as transformações aplicadas a um grupo sejam herdadas corretamente pelos subgrupos, de forma acumulada ao longo da árvore de cena.

Para garantir que as transformações de um grupo não afetem outros grupos dentro da cena, utilizamos os comandos `glPushMatrix` e `glPopMatrix`.

3. Testes

Nesta fase, foram realizados vários testes para validar a aplicação das transformações geométricas na cena. Os testes incluem translações, rotações e escalas aplicadas a diferentes modelos, garantindo que as transformações hierárquicas funcionam corretamente.

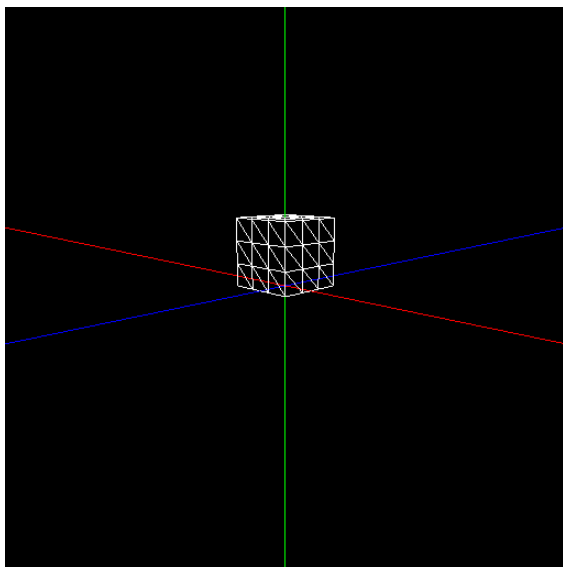


Figura 4: Teste 1 - Cubo com translação

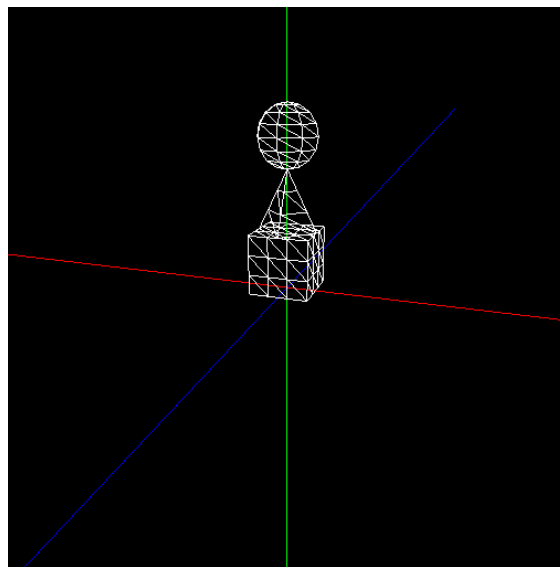


Figura 5: Teste 2 - Cubo, cone e esfera com translações

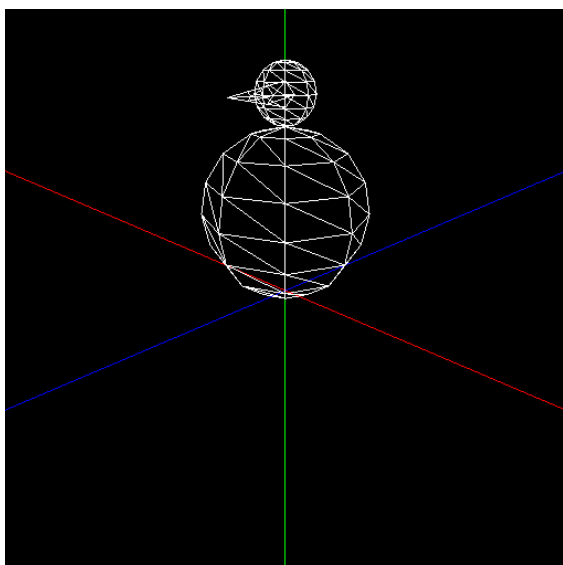


Figura 6: Teste 3 - Duas esferas e um cone com translações, escalas e rotações

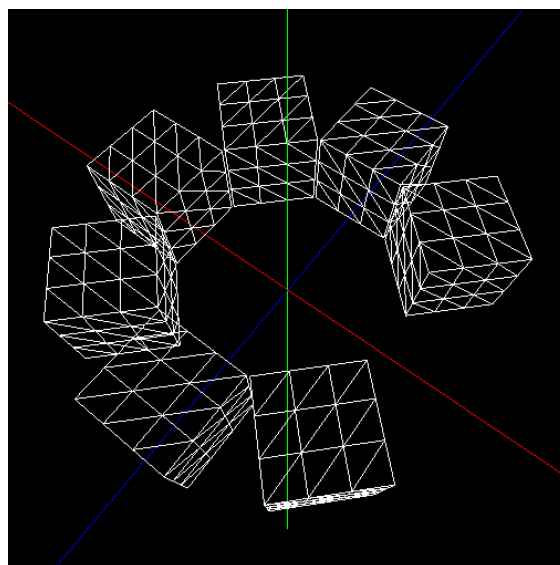


Figura 7: Teste 4 - Cubos com translações e rotações

Além dos testes realizados anteriormente, foi desenvolvida uma nova cena que representa o **Sistema Solar**, contendo o Sol, os planetas e seus respectivos satélites:

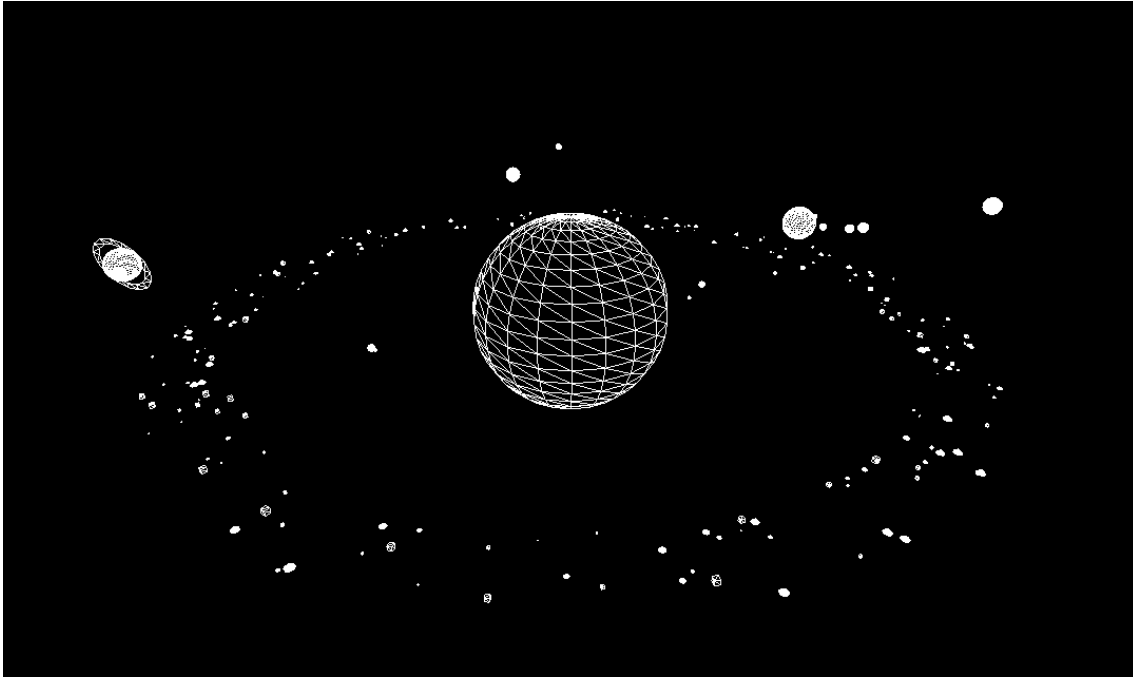


Figura 8: Renderização da cena do sistema solar

A criação da cena foi feita utilizando uma **estrutura hierárquica no XML**, onde cada planeta é representado como um grupo com as suas próprias transformações geométricas, e os seus subgrupos contêm as componentes correspondentes, como luas e anéis. Para gerar o ficheiro *XML*, foi desenvolvido um gerador que cria a cena de forma aleatória, proporcionando variação nas configurações do Sistema Solar.

A hierarquia segue o princípio de que cada grupo herda as transformações do seu **grupo pai**, permitindo que luas orbitem os seus planetas corretamente. Por exemplo, a **Lua** é um subgrupo da **Terra**, enquanto **Fobos** e **Deimos** são subgrupos de **Marte**. Além das esferas para representar planetas e luas, utilizamos o modelo adicional do toro para os anéis de Saturno, e cones e cubos para representar os asteroides.

Para recriar a cena (já disponível), execute o comando abaixo dentro da pasta `build` :

```
./generator solar
```

Caso deseje apenas gerar os modelos necessários para visualizar a cena, use os seguintes comandos no terminal do diretório `build` :

```
./generator torus 1.7 0.2 2 15 torus.3d  
./generator cone 1 1 20 5 cone.3d  
./generator sphere 1 20 20 sphere.3d  
./generator box 1 1 box.3d
```

Com o ficheiro *XML* e modelos gerados, pode visualizar a cena executando o comando abaixo dentro da pasta `build` :

```
./engine ../src/scenes/ours_tests/solar_system.xml
```

4. Scripts

Na segunda fase, foram adicionados novos *scripts* de teste para validar a renderização de cenas hierárquicas e a aplicação correta das transformações geométricas. Esses *scripts* seguem o mesmo princípio dos anteriores, executando a geração dos modelos necessários e inicializando o engine com os respectivos ficheiros XML de teste desta fase.

Os novos *scripts* são:

1. **test1_p2.sh**: Teste 1 de Cubo com translação.
2. **test2_p2.sh**: Teste 2 de Cubo, cone e esfera com translações.
3. **test3_p2.sh**: Teste 3 de Duas esferas e um cone com translações, escalas e rotações.
4. **test4_p2.sh**: Teste 4 de Cubos com translações e rotações.
5. **solar_system.sh**: Teste adicional com cena do sistema solar.

Estes *scripts* foram criados para facilitar a verificação da implementação das transformações hierárquicas e garantir que o **Engine** interpreta corretamente a nova estrutura do XML.

Para executar os *scripts* de teste, siga os passos abaixo em um ambiente **Linux**.

Navegue até a pasta raiz do projeto e execute os comandos no terminal:

- Para partilha das permissões de execução - `chmod -R +x scripts/`
- Para execução do teste - `./scripts/phase2/test`

5. Conclusões

Nesta fase, conseguimos implementar todos os requisitos propostos, atendendo às necessidades de transformações geométricas hierárquicas para a criação de cenas complexas. A solução foi validada através de testes práticos, confirmando que a hierarquia e a aplicação das transformações funcionam conforme esperado, sem necessidade de mudanças adicionais. Com isso, estabelecemos uma base sólida para a próxima fase, onde iremos trabalhar com curvas, superfícies cúbicas e otimização da renderização com VBOs.